

EO-SAR Binary Change Detection

Technical Report

This file is the consolidated submission report. It merges the important material previously split across `reports/technical_report_draft.md` , `reports/time_resource_log.md` , `reports/metrics_table_debug.md` , and the earlier `reports/final_technical_report.md` draft into one PDF-ready source document.

1. Abstract

This project addresses binary change detection on paired Electro-Optical (EO) and Synthetic Aperture Radar (SAR) satellite imagery. The task is pixel-wise classification of each location into `No-Change` or `Change` using a pre-event EO image and a post-event SAR image. The implemented baseline is a dual-encoder EO-SAR UNet with separate ResNet34 backbones, multi-scale feature fusion, and an attention-guided decoder. The repository now contains a complete PyTorch pipeline for data loading, training, evaluation, visualization, and notebook-based qualitative inspection. The best recorded validation result from the completed full run is IoU `0.5284` with validation F1 approximately `0.69` , which provides a credible baseline for this multi-modal change detection task.

2. Project Summary

This project addresses binary change detection on paired Electro-Optical (EO) and Synthetic Aperture Radar (SAR) satellite imagery. The task is pixel-wise classification of each location into `No-Change` or `Change` using a pre-event EO image and a post-event SAR image.

The implemented baseline is a dual-encoder UNet-style segmentation network with modality-specific encoders, multi-scale fusion, attention-guided decoding, and a complete PyTorch training/evaluation workflow.

3. Problem Definition

- Input: pre-event EO RGB image and post-event SAR grayscale image

- Output: binary change mask
- Original labels: 0=Background , 1=Intact , 2=Damaged , 3=Destroyed
- Required remapping: {0,1} -> 0 and {2,3} -> 1

4. Dataset

- Train samples: 2781
- Validation samples: 334
- Test samples: 77
- Directory structure: dataset/{train,val,test}/{pre-event,post-event,target}

5. Technical Challenges

- Multi-modal fusion of EO and SAR data
- Strong class imbalance where change pixels are minority class
- Dense pixel-level prediction instead of image-level classification
- Generalization across disaster scenes with limited labeled data

6. Literature and Design Rationale

- Siamese and pseudo-Siamese encoder-decoder architectures are strong baselines for change detection because they preserve modality-specific representations while still enabling comparison across paired inputs.
- EO-SAR fusion requires modality-aware processing because optical and radar imagery differ in channel layout, noise characteristics, and physical sensing properties.
- Severe foreground imbalance makes overlap-aware and hard-example-aware losses such as Focal + Dice more appropriate than plain cross-entropy alone.
- A UNet-like decoder remains a strong choice for dense prediction because it recovers spatial detail through skip connections while preserving high-level semantics from deeper encoder stages.

7. Data Pipeline

Implemented in `src/data/dataset.py` .

- TIFF loading for EO, SAR, and masks
- Filename alignment checks across all three folders
- EO validated as 3-channel RGB, SAR as single-channel grayscale, target as 2D mask

- Label remapping applied before training and evaluation
- Resize to 256 x 256
- Bilinear interpolation for EO/SAR and nearest-neighbor for masks
- Separate normalization for EO and SAR
- Joint augmentations: random flips, 90-degree rotations, EO-only color jitter

Normalization values from config.yaml :

- EO mean: [0.485, 0.456, 0.406]
- EO std: [0.229, 0.224, 0.225]
- SAR mean: [0.5]
- SAR std: [0.25]

8. Model Architecture

Implemented in src/models/siamese_unet.py as DualEncoderEOSARUNet .

- EO encoder: pretrained ResNet34 with 3-channel input
- SAR encoder: pretrained ResNet34 adapted to 1-channel input
- SAR first convolution initialized by averaging pretrained RGB weights
- Multi-scale EO and SAR features concatenated at each encoder stage
- Attention-gated UNet decoder with transposed-convolution upsampling
- Output head produces 2-channel logits for binary segmentation

Why this architecture was chosen:

- Separate encoders preserve modality-specific features
- Multi-scale fusion helps detect both small and large change regions
- Attention gates help suppress irrelevant skip features
- UNet-style decoding is a strong segmentation baseline

9. Loss, Optimization, and Training Setup

Implemented in src/utils/losses.py , src/train.py , and config.yaml .

- Loss: FocalDiceLoss
- Focal weight: 0.5
- Dice weight: 0.5
- Focal gamma: 2.0
- Optimizer: AdamW
- Learning rate: 1e-4
- Weight decay: 1e-4

- Scheduler: cosine annealing
- Epoch budget: 50
- Early stopping patience: 10
- Minimum delta: 0.001
- Batch size: 16
- Image size: 256
- AMP: false
- Seed: 42
- Device used in completed run: cpu

Implemented training features:

- Deterministic seeding
- Config-driven dataloaders
- Epoch-wise metric logging
- Best checkpoint saving to checkpoints/best.pth
- Early stopping based on validation IoU

10. Evaluation Protocol

Implemented in src/eval.py and src/utils/metrics.py .

- Metrics reported for the change class: IoU, precision, recall, F1 score
- Confusion matrix is also saved in the metrics output
- For 2-channel logits, softmax is applied and change probability is thresholded at 0.5

11. Visualization and Notebook Support

- src/visualize.py generates qualitative prediction grids
- notebooks/evaluate_validation_checkpoint.ipynb evaluates checkpoints/best.pth on validation data
- The notebook samples random validation examples on each run and shows target vs prediction
- The notebook also prints whether each sample is a Correct prediction or False prediction and shows per-sample accuracy, precision, recall, F1, and IoU

12. Experimental Results

12.1 Final Training Run

From `logs/training.log` :

- Device: `cpu`
- Train samples: `2781`
- Validation samples: `334`
- Early stopping triggered after `45` epochs
- Best validation IoU: `0.5284`
- Best checkpoint: `checkpoints/best.pth`

Final epoch metrics:

- Train loss: `0.1201`
- Train IoU: `0.5637`
- Train precision: `0.7740`
- Train recall: `0.6748`
- Train F1: `0.7210`
- Validation loss: `0.1863`
- Validation IoU: `0.5260`
- Validation precision: `0.7125`
- Validation recall: `0.6676`
- Validation F1: `0.6894`

12.2 Validation Evaluation Metrics

From `results_val_final/evaluation_metrics.json` using `checkpoints/best.pth` :

- IoU: `0.5284`
- Precision: `0.7486`
- Recall: `0.6425`
- F1: `0.6915`
- Confusion matrix: `[[21303503, 103929], [172186, 309406]]`

12.3 Test Evaluation Metrics

From `results_test_final/evaluation_metrics.json` using `checkpoints/best.pth` :

- IoU: `0.0039`
- Precision: `0.0358`
- Recall: `0.0044`
- F1: `0.0078`
- Confusion matrix: `[[5003624, 4527], [37953, 168]]`

12.4 Interpretation

- The model gives a solid baseline for EO-SAR binary change detection
- Validation precision is stronger than recall, so the model is relatively selective but still misses some changed pixels
- The train/validation gap is moderate, indicating useful generalization on the validation split with room for tuning
- The held-out test results are much weaker than the validation results, which suggests a substantial domain shift or generalization failure on the available test subset
- This makes the current model suitable as a baseline submission, but it also shows that stronger regularization, threshold tuning, or improved fusion/training strategy is still needed

12.5 Debug Verification Metrics

Before the full training run, a short debug experiment was used to verify the end-to-end pipeline with `checkpoints_debug/best.pth`.

Split	IoU	Precision	Recall	F1
Validation (debug)	0.0220	0.0220	1.0000	0.0431
Test (debug)	0.0076	0.0076	1.0000	0.0150

Interpretation of the debug run:

- The debug checkpoint over-predicted the change class, giving very high recall and very poor precision.
- These numbers were useful only for pipeline verification and should not be treated as final model performance.

12.6 Figures and Supporting Artifacts

- Final validation predictions: `results_val_final/predictions.png`
- Final test predictions: `results_test_final/predictions.png`
- Final validation metrics: `results_val_final/evaluation_metrics.json`
- Final test metrics: `results_test_final/evaluation_metrics.json`
- Debug validation predictions: `results_debug_val/predictions.png`
- Debug test predictions: `results_debug_test/predictions.png`
- Debug validation confusion matrix: `results_debug_val/confusion_matrix.png`
- Debug test confusion matrix: `results_debug_test/confusion_matrix.png`

- Debug summary: `results_debug_summary.md`

13. Important Implementation Notes

- Binary label remapping is correctly enforced in the dataset pipeline
- EO and SAR are normalized separately to reflect different data distributions
- `invalid_mask` is computed in the dataset class but not yet used in loss or metrics
- `model.backbone` is present in config, but the implemented model path currently corresponds to the ResNet34-based architecture
- Main CLI evaluation reports IoU, precision, recall, and F1, not plain pixel accuracy

14. Limitations

- CPU training makes experimentation slow
- No threshold tuning or class-balanced sampling has been added yet
- Nodata-aware masking is not yet used during optimization
- Backbone exploration beyond the current ResNet34 baseline is still open
- Test-set generalization is currently poor relative to validation performance

15. Future Work

- Tune the change threshold instead of fixing it at `0.5`.
- Explore stronger imbalance strategies such as class-balanced sampling or tuned focal/dice weights.
- Compare the current ResNet34 baseline against deeper encoders or improved fusion blocks.
- Add nodata-aware masking directly into optimization and metric computation if needed.
- Evaluate larger input sizes or patch-based higher-resolution training if compute resources allow.

16. Remaining Submission Items

- No remaining blocked items in the local submission package.
- The public submission ZIP link can be provided directly in the submission form.

17. Submission Artifacts

- Source code repository
- `config.yaml`
- `requirements.txt`
- `README.md`
- `checkpoints/best.pth`
- `results_val_final/evaluation_metrics.json`
- `results_test_final/evaluation_metrics.json`
- `results_val_final/predictions.png`
- `results_test_final/predictions.png`
- Final report PDF
- Public GitHub repository: <https://github.com/rayhanakhtar/galaxeye-eo-sar-change-detection>

18. Time and Resource Log

18.1 Development Work Completed

- Dataset inspection and modality verification
- Literature survey and baseline selection
- Dataset class implementation and label remapping validation
- Dual-encoder model implementation
- Loss, metrics, evaluation, and visualization utilities
- Training pipeline with logging, checkpointing, scheduler, and early stopping
- Validation notebook setup and kernel/environment fixes
- Random-sample qualitative prediction cell added to the evaluation notebook

18.2 Compute Resources Used

- Local environment: Python virtual environment on Windows
- Framework: PyTorch
- Device used for completed full run: CPU
- CUDA availability during local validation work: `False`
- Mixed precision: disabled
- Batch size: `16`
- Image size: `256 x 256`
- Earlier debug verification used `debug_config.yaml` with 4 samples and 1 epoch

18.3 Training Time Log

Based on `logs/training.log` , the full CPU run started at about `2026-05-13 00:47:58` and ended at `2026-05-13 21:57:55` .

- Approximate wall-clock training time: `21 hours 10 minutes`
- Epochs completed: `45`
- Stop condition: early stopping

18.4 Output Artifacts

- Best checkpoint: `checkpoints/best.pth`
- Training log: `logs/training.log`
- Validation metrics JSON: `results_val_final/evaluation_metrics.json`
- Test metrics JSON: `results_test_final/evaluation_metrics.json`
- Validation visualization: `results_val_final/predictions.png`
- Test visualization: `results_test_final/predictions.png`
- Validation notebook: `notebooks/evaluate_validation_checkpoint.ipynb`

19. Conclusion

This project delivers a complete and reproducible EO-SAR binary change detection baseline with correct label remapping, modality-aware preprocessing, class-imbalance-aware optimization, checkpointing, evaluation, visualization, and notebook-based inspection tools.

The best recorded validation IoU is `0.5284` and validation F1 is about `0.69` , but the current held-out test metrics are much lower, with test IoU `0.0039` and test F1 `0.0078` . The project is still technically complete as a baseline submission, and the final report now captures both the strengths of the implemented pipeline and the present generalization gap that should be addressed in future improvement work.